

Coupled Energy-Minimization Smoothing and Topological Mesh Adaptation: Design of an Adaptivity Loop for Norma

Alejandro Mota

August 2026, revised September 2026

Abstract

Energy-minimization smoothing (EMS) improves a mesh by minimizing a hyperelastic distortion energy over the nodal positions, with each element measured against an ideal reference element. Smoothing alone, however, terminates at configurations where no continuous motion of the nodes can further reduce the energy, while the mesh remains poor because the obstruction is topological. Numerical experiments show that alternating EMS with targeted topological operations—edge swaps, collapses, and splits applied to the elements of highest distortion energy—improves mesh quality far beyond what either mechanism achieves in isolation. This note designs such a coupled loop for Norma, informed by a detailed reading of the adaptation logic of the Omega.h library. Three questions are addressed with concrete algorithms: when to trigger topological operations, how to guarantee a monotonic decrease of the total distortion energy across both smoothing and topological phases, and how to detect stagnation of the smoothing phase with a scale-invariant criterion. The central design decision is to use a single distortion functional, well defined for any mesh topology, as the objective of both phases: with a prescribed size or metric field, the metric-conditioned smoothing energy now in Norma is such a functional; without one, an intrinsic equal-volume functional takes its place. The revision of September 2026 fixes the scope (a preprocessing step on four-node tetrahedral meshes, with no transfer of solution state), records the alternatives of wrapping Omega.h or keeping the hand-off to the Sierra tool `improve_mesh`, and states why the topological operations should be accepted on the energy with geometric quality as a floor. The note closes with a step-by-step implementation plan, recommendations for topology and adjacency data structures, and a ranked list of risks.

Contents

1	Motivation and observed workflow	2
2	Existing infrastructure in Norma	3
2.1	EMS as a solid-mechanics solve	3
2.2	Assets already in place	3
2.3	Gaps	4
3	The adaptation logic of Omega.h	4
3.1	Two tiers of thresholds	4
3.2	Control flow	4
3.3	Termination	5

4	The Sierra hand-off and the alternatives to a native loop	5
5	A single distortion functional	7
5.1	The obstruction	7
5.2	With a prescribed size or metric field	7
5.3	Without a prescribed target: the equal-volume functional	8
5.4	Re-baselining	8
6	Question 1: adaptivity triggers	8
7	Question 2: energy monotonicity	9
7.1	The acceptance gate	9
7.2	The smoothing phase	10
8	Question 3: stagnation detection	11
9	Architectural plan	12
10	Data structures and design patterns	13
11	Risks and open questions	14
12	Summary	14

1 Motivation and observed workflow

Mesh smoothing by energy minimization treats the mesh as a hyperelastic body whose per-element reference configuration is an ideal element, and minimizes the resulting stored energy over the nodal positions. The energy acts as a barrier against element inversion and drives every element toward the ideal shape. The mechanism is purely geometric: the connectivity is fixed, and only the nodal positions move.

This fixity is also the limitation. Many poor meshes contain configurations—slivers wedged between well-shaped neighbors, vertices of excessive valence, edges shared by too many or too few elements—whose distortion cannot be removed by any continuous motion of the nodes. Smoothing then stalls: the energy plateaus, or the line search fails against the inversion barrier while the gradient remains large. The obstruction is topological, and the remedy is a local change of connectivity. Conversely, topological operations applied to an unsmoothed mesh are evaluated at poor nodal positions and are frequently rejected or mis-ranked. The two mechanisms are complementary, and the literature on tetrahedral mesh improvement confirms that their combination is substantially stronger than either alone [3, 4].

The workflow this note develops is therefore iterative:

1. run EMS until the energy decrease stagnates;
2. identify the elements of highest distortion energy;
3. apply topological operations to the neighborhoods of those elements, accepting only operations that decrease the energy;
4. resume EMS, which the changed topology now permits to relax further;
5. repeat until neither mechanism makes progress.

Scope. The loop is a preprocessing step: it produces a mesh before any analysis is run on it, so no solution state exists on the mesh and no state transfer is involved. It targets four-node tetrahedra only, which is the element type the smoothing supports and the type the topological operations are written for; a mesh of quadratic tetrahedra is obtained afterwards by conversion, placing the midside nodes at the edge midpoints and projecting those on the boundary to the surface with the level-set machinery already in place. The workflow it replaces is staggered: Norma smooths, the smoothed mesh is passed to the Sierra tool `improve_mesh`, which changes the topology against a geometric quality measure, and the result may be smoothed again (Section 4).

The design questions are precisely where the coupling is delicate: the trigger for step 3, the acceptance criterion that keeps the total energy monotone across steps 1–4, and the stagnation test in step 1. Sections 6–8 address them in turn, after three preparatory sections: an inventory of the machinery Norma already possesses (Section 2), a summary of the decision logic of Omega.h as implemented in its source code (Section 3), and an assessment of the alternatives to a native implementation (Section 4).

2 Existing infrastructure in Norma

2.1 EMS as a solid-mechanics solve

Norma implements EMS not as a separate module but as a mode of the ordinary quasi-static `SolidMechanics` solve. A single substitution in the element loop (`src/model.jl`) replaces each element’s reference configuration by an ideal regular tetrahedron before computing the deformation gradient; everything else—assembly, solvers, boundary conditions, output—is the standard machinery. The energy is the Seth–Hill family with exponents $m = n = 2$, which diverges both as the element Jacobian tends to zero and as it tends to infinity, and therefore acts as an inversion barrier. The nodal positions are the unknowns; the minimization runs under the quasi-static integrator as pseudo-time continuation, with matrix-free steepest-descent or L-BFGS steps.

The ideal reference element is built by `create_smooth_reference` (restricted at present to four-node tetrahedra) as a regular tetrahedron of edge length h , where h follows one of four rules: equal volume to the original element, average edge length of the original element, the maximum of these two, or a user-supplied size field $s(t, x, y, z)$. A fifth rule, `metric field` (September 2026), replaces the scalar size by a metric tensor with three principal sizes and a rotation; the ideal element is then anisotropic and the energy is evaluated on the deformation gradient measured in the metric, so that the orientation of the elements enters the objective (`docs/notes/ems-anisotropic`). In all rules the input geometry is the *original* mesh as read from disk; the consequences of this choice for adaptivity are the subject of Section 5.

Boundary nodes move but remain on the boundary, through either axis-aligned Dirichlet conditions or the `Surface` boundary condition, which represents a boundary surface as an analytic level set $g(\mathbf{x}) = 0$ and enforces it by projection: nodes are returned to the surface before assembly (`return_to_surface!`) and the gradient is projected onto the tangent space afterward. This machinery is directly reusable for placing and constraining nodes during topological operations on curved boundaries.

2.2 Assets already in place

Three existing components carry much of the weight of the proposed loop.

1. *A per-element distortion field.* The integrated element energy is stored per block and element in `model.stored_energy`, updated on every evaluation of the model and already written to

the Exodus output as an element variable. Step 2 of the workflow—finding the most distorted elements—requires no new computation.

2. *Cross-mesh state transfer.* The module `src/field_transfer.jl` performs a consistent L^2 transfer of nodal fields between meshes of the same geometry, and composes it with the variational recovery of [6] to remap integration-point internal variables. The preprocessing loop of this note does not need it; it is what a future in-simulation adaptivity capability would require, and it connects to the constrained projection of the companion note on internal-variable recovery in admissible spaces (`docs/notes/internal-variable-projection`).
3. *Mid-run mesh replacement.* The module `src/swap.jl` already replaces the mesh of a running simulation: it rebuilds the model, integrator, and solver, and transfers the state to the new mesh. This is the orchestration pattern a topology change must follow, for a structural reason: `initialize_storage` aliases the integrator and solver arrays into the model matrices through raw pointers (`unsafe_wrap`), so any reallocation of the model arrays invalidates the integrator and solver. A topology change must therefore proceed by full rebuild, never by in-place resizing.

2.3 Gaps

Equally important is what does not exist. The mesh is never held in memory as a mutable object: the `ExodusDatabase` handle is the source of truth, connectivity is re-read from the file on every access, and no function writes coordinates or connectivity back. There are no adjacency structures of any kind—no node-to-element, element-to-element, or edge data—so there is at present no way to enumerate the cavity around an edge. There is no element quality module; the de facto quality measure of the code base is the smoothing energy itself. (Point location, a linear scan when this note was first written, is now a uniform-grid search in `src/spatial_search.jl`.) These gaps define Step 3 of the implementation plan in Section 9.

3 The adaptation logic of Omega.h

`Omega.h` [1, 2] is a metric-driven simplex-adaptation library whose decision logic is compact and, as verified directly against its source code, organized around three ideas: a two-tier system of thresholds, cavity operators filtered by acceptance criteria, and progress-based termination. This section records the logic precisely, because Section 6 transplants it.

3.1 Two tiers of thresholds

The options structure `AdaptOpts` carries *desired* and *allowed* values with distinct roles: desired values select candidates for work, allowed values are hard gates that no accepted operation may violate. The defaults are: metric edge lengths desired within $[1/\sqrt{2}, \sqrt{2}]$, length allowed up to $2\sqrt{2}$; element quality desired at least 0.30 (three dimensions), quality allowed at least 0.20. There is no lower allowed length: short edges are only ever “desired” away. The quality measure is the metric-conditioned mean ratio, an intrinsic shape measure equal to one for a metric-equilateral simplex and non-positive for a degenerate or inverted one.

3.2 Control flow

The top-level `adapt()` first evaluates an “already satisfactory” predicate—minimum quality at least the desired quality *and* all metric edge lengths within the desired band—and returns without

touching the mesh if it holds. Otherwise it runs two phases.

1. *Length phase.* Alternate refinement of edges longer than $\sqrt{2}$ and collapse of edges shorter than $1/\sqrt{2}$, repeating until neither operator changes anything. Acceptance in this phase enforces only the hard gates: a refinement is accepted if the worst new element stays above the allowed quality; a collapse additionally may not create edges beyond the allowed length. Quality may therefore *decrease* in this phase, down to the floor.
2. *Quality phase.* While the minimum quality is below the desired value: attempt edge swaps; only if a full swap pass changes nothing, attempt sliver collapses; if both fail in the same pass, report failure and stop. Acceptance in this phase is strict improvement: a swap or sliver collapse is kept only if the minimum quality of the new cavity *strictly exceeds* that of the old cavity. This strict filter is the only monotonicity guarantee in the library.

Candidates for the quality phase are the elements below desired quality *dilated by four layers of adjacent elements*, because the operation that repairs a bad element frequently lies on a neighboring entity. Boundary edges are never swapped. Conflicts between overlapping cavities are resolved by a Luby-style independent set in which the cavity of higher quality wins, with a deterministic global-identifier tiebreak; candidates not selected simply wait for the next pass.

3.3 Termination

There is no iteration counter anywhere in `adapt()`. Every operator returns false exactly when no candidate survives its filter chain, and each phase loop exits when its operators return false. Termination is thus purely progress-based: the loop runs precisely as long as some operation can be accepted. When the quality goal is unreachable, the library says so and returns the mesh in its best achieved state rather than looping.

4 The Sierra hand-off and the alternatives to a native loop

Three routes exist: keep the hand-off to `improve_mesh`, wrap `Omega.h` into Norma, or implement the operators natively. The facts about the two external tools, read from their sources in September 2026, are these.

improve_mesh and emend. `improve_mesh` is a thin driver (about 2,500 lines: an input parser, STK mesh creation, and optional CAD geometry through `m2g`) over the Sierra library `emend` (93 files, about 10,000 lines). `emend` implements edge swaps, face swaps, edge collapses (with a variant for mandatory node removal), edge refinement, independent-set selection of non-conflicting cavities, control of the deviation from the geometry (`edgeGeomDeviationTol`, `sharpFeatureAngleTolInDegrees`), and midnode straightening for quadratic elements. Its parameters follow the `Omega.h` logic: desired and allowed relative edge lengths at $1/\sqrt{2}$ and $\sqrt{2}$, a quality below which improvement is attempted, iteration caps per operator, and termination when the quality stops improving. It accepts nodal isotropic and anisotropic size fields, Cartesian-aligned and cylindrical size fields, and a displacement field. Its quality is an abstract `QualityMetric` compared through `is_first_quality_metric_better_than_second`, so the measure is pluggable, but acceptance is always the comparison of the worst element of a cavity before and after the operation. It contains no field transfer and no code from `Omega.h`: it is an independent implementation of the same decision logic on STK, and it is not open source.

Omega.h. C++14 with CMake; optional MPI, OpenMP, or CUDA; tetrahedra and triangles; anisotropic metric fields with gradation control; refinement, coarsening, swapping, and sliver coarsening under `AdaptOpts`; Exodus input and output through SEACAS; boundary geometry through EGADS or through entity classification alone. The quality is the metric-conditioned mean ratio, fixed in the library; the one extension hook, `UserTransfer`, concerns fields, not the acceptance criterion. There is no C interface.

Comparison. Table 1 sets the two routes that would bring the operations into Norma against each other.

	Wrap Omega.h	Native operators in Julia
Work to first result	A C shim (mesh arrays in and out, metric per vertex, options, the <code>adapt</code> call) of a few hundred lines plus a <code>ccall</code> layer, packaged as an optional extension; about two weeks	Steps 3 to 7 of Section 9; three to four thousand lines of Julia with tests; two to three months to a validated loop
Build and distribution	A C++ dependency with CMake in a code that runs wherever Julia runs; CI on Ubuntu and macOS must build it; a binary recipe for users	None
Acceptance criterion	Fixed: metric-conditioned mean ratio and the length band; an energy criterion requires patching the library	Any: the energy gate of Section 7, a geometric floor, or both
Coupling with smoothing	Loose: the two tools optimize different objectives	Tight: one objective, alternating phases, monotone descent
Boundary geometry	Classification only, or EGADS; the analytic <code>Surface</code> level sets would have to be re-expressed	<code>Surface</code> level sets and node and side sets reused directly
Parallelism	MPI and threads	Serial at first; threads later through independent sets
Maturity	High, deterministic, with stated quality guarantees	To be earned: cavity validity, orientation, and boundary preservation need careful tests
Public repository	Yes (BSD license)	Yes

Table 1: Bringing topological operations into Norma: wrapping Omega.h against a native implementation.

Assessment. Keeping the hand-off is mature and in use, but it cannot be part of a public Norma workflow, and the two tools will always optimize different quantities (the energy in Norma, a geometric quality in `emend`), which is the source of the inefficiency of the staggered loop. Wrapping Omega.h buys a mature adapter quickly but reproduces that staggered loop with a different external tool and adds a build dependency. The value of doing this inside Norma is one objective for both phases, and that requires the native route. The native route is a bounded piece of work because its hard parts exist or are specified: the energy kernel and the metric-space ideal element are in the code, the geometry constraints exist, the orchestration pattern exists, and the operators for four-node tetrahedra are well documented (an edge loop of at most seven elements admits a small

table of triangulations; collapses and splits are local). `emend` is ten thousand lines because it also handles quadratic elements, CAD geometry, STK, and parallel independent sets, none of which are in the first version here. The recommendation is the native route, with `Omega_h` in one narrow role: its command-line tools read and write Exodus, so a mesh and a metric can be run through it and its length and quality histograms compared with ours at no integration cost, and with `improve_mesh` as the reference for the validation cases of Section 9.

5 A single distortion functional

5.1 The obstruction

The design problem that governs the entire coupling is that the smoothing objective under the `equal volume`, `average edge length`, and `max` rules is tied to the original mesh: the ideal reference of each element is built from the geometry of that element in the mesh as read from disk. After a topological operation, a new element has no original counterpart, so the objective is undefined on it; and even where defined, an energy computed against history-dependent targets cannot be compared across a topology change, because the comparison would be between different functionals. A monotonicity guarantee requires a single functional, defined for every admissible pair of topology and nodal positions, that both phases of the loop demonstrably decrease. There are two such functionals, depending on whether a target is prescribed.

5.2 With a prescribed size or metric field

When the target is a size field $h(\mathbf{X})$ or a metric field $\mathbf{M}(\mathbf{X})$ (`docs/notes/ems-anisotropic`), the ideal element of any tetrahedron is the metric sampled at its centroid, for a new element exactly as for an old one. The smoothing energy

$$D(\mathcal{T}, \mathbf{x}) = \sum_{e \in \mathcal{T}} V_{\mathbf{X}}^e \psi(\mathbf{F}_{U,e}), \quad \mathbf{F}_{U,e} = \mathbf{F}_M \mathbf{F}_e \mathbf{F}_M^{-1}, \quad (1)$$

with $V_{\mathbf{X}}^e = h_1 h_2 h_3 V_{\text{unit}}$ the volume of the ideal element and $\mathbf{F}_{U,e}$ the deformation gradient measured in the metric, is therefore defined for every admissible pair $(\mathcal{T}, \mathbf{x})$ without reference to the history of the mesh. It measures size, shape, and orientation against the target in one number: it vanishes only for a unit element of the metric, it diverges at inversion, and it reduces to the isotropic energy for equal sizes. This is the functional the loop should use whenever a target is prescribed, and it changes two things in what follows. First, an operation that moves the mesh toward the target size is a decrease of the same functional the smoother minimizes: a too-large element has a large volumetric energy and its children, each nearer the target volume, have less; so size-driven splits and collapses need no separate phase and no exemption from the acceptance gate, and the length band of `Omega_h` survives only as the inexpensive way to select which edges to try. Second, the volume floor of the rules `size field` and `metric field`, which scales the target up to the volume of the original element, exists only because smoothing cannot change the number of elements; in the coupled loop it is unnecessary and the `unrestricted` variants are the right ones. The target is held fixed during a smoothing phase, sampled at the centroids of the elements at the start of the phase, and re-sampled at every phase boundary, as the anisotropic note describes; a new element created by a topological operation receives the metric at its centroid when it is created. For thresholds the natural density is $\hat{w}_e = \psi(\mathbf{F}_{U,e})$, the energy per unit ideal volume, which is dimensionless with unit moduli.

5.3 Without a prescribed target: the equal-volume functional

Without a target, the mesh should keep its local size and only improve its shape. Let $\mathcal{T}$ denote the mesh topology and $\mathbf{x}$ the nodal positions. Define

$$D(\mathcal{T}, \mathbf{x}) = \sum_{e \in \mathcal{T}} W_e(\mathbf{x}), \quad W_e(\mathbf{x}) = \int_{\Omega_e^{\text{ideal}}} \psi(\mathbf{F}_e) dV, \quad (2)$$

where ψ is the Seth–Hill energy density already in use, $\mathbf{F}_e$ is the deformation gradient of element e measured from its ideal reference, and—this is the decision—the ideal reference of each element is the regular tetrahedron of *equal volume to that element’s own current volume*. This is the existing "equal volume" rule of `create_smooth_reference`, applied to the current rather than the original geometry.

With this choice the energy of an element depends only on its own present shape: it is invariant under uniform scaling, vanishes exactly for a regular tetrahedron, and diverges at degeneracy. The functional (2) is therefore *intrinsic*—a function of $(\mathcal{T}, \mathbf{x})$ alone, with no dependence on mesh history—and is well defined for any tetrahedral mesh whatsoever. It belongs to the same class as the mean-ratio quality of `Omega_h`, which is likewise a scale-invariant shape measure of the current element; the difference is that (2) is an energy with an inversion barrier rather than a bounded quality, which is precisely what makes it minimizable by the existing solvers. For threshold purposes define the normalized density

$$\hat{w}_e = \frac{W_e}{V_e}, \quad (3)$$

with V_e the current element volume, so that $\hat{w}_e = 0$ for an ideal element and $\hat{w}_e \rightarrow \infty$ at degeneracy. Note the orientation: minimum quality in `Omega_h` corresponds to *maximum* energy density here.

Remark 1. *Size-targeting is excluded from (2) by construction: this functional keeps whatever size the mesh has. Operations toward a target size belong to the functional (1) of Section 5.2, where they are decreases of the objective and need no separate phase. The two functionals are not mixed within one loop: a run either has a prescribed target or it does not.*

5.4 Re-baselining

Re-baselining concerns the equal-volume functional; with a prescribed target the ideal element is already fixed by the field. The smoothing solver requires a *fixed* objective during a solve: if the ideal reference tracked the current volume continuously, the target would move with the unknowns. The resolution is re-baselining at phase boundaries: at the start of each smoothing phase, set the smoothing reference configuration to the current coordinates and build the equal-volume ideal references from it. The phase objective then coincides with $D(\mathcal{T}, \cdot)$ at the start of the phase and remains a fixed, minimizable functional throughout it. The relation between the frozen-target phase objective and the intrinsic functional is that of a majorize–minimize surrogate; the safeguard of Proposition 1 converts the practical descent of D into a guaranteed one.

6 Question 1: adaptivity triggers

The translation of the `Omega_h` logic into the energy setting is summarized in Table 2 and specified below.

Omega_h	Proposed EMS loop
mean-ratio quality (max. 1, degenerate ≤ 0)	energy density $\hat{w}_e$ (min. 0, degenerate $\rightarrow \infty$)
<code>min_quality_desired</code> (candidate selection)	$\hat{w}_{\text{des}}$: element is worth working on
<code>min_quality_allowed</code> (hard floor)	$\hat{w}_{\text{all}} > \hat{w}_{\text{des}}$: hard ceiling
metric length band $[1/\sqrt{2}, \sqrt{2}]$	same band selects split and collapse candidates
<code>nsilver_layers</code> = 4 adjacency layers	same: dilate candidate set by n_{layers} rings
swap first, sliver-collapse as fallback	same order
strict cavity-quality improvement filter	strict cavity-energy decrease (Section 7)
progress-based termination, no counters	same

Table 2: Correspondence between the Omega_h adaptation logic and its proposed energy-based analogue. The orientation inverts: minimum quality corresponds to maximum energy density.

1. *Trigger for the topology phase.* Enter the topology phase when the smoothing phase has stagnated in the sense of Section 8 and $\max_e \hat{w}_e > \hat{w}_{\text{des}}$. This is the “already satisfactory” predicate of Omega_h with quality replaced by energy density; with a prescribed target the energy density already contains the length clause, since an element of the wrong size has a large volumetric energy.
2. *Candidate set.* All elements with $\hat{w}_e > \hat{w}_{\text{des}}$, dilated by n_{layers} rings of element adjacency (default four, following Omega_h); the operation candidates are the interior edges of this set. Boundary edges are excluded from swapping; they participate in collapses and splits only under the surface constraints of Section 9.
3. *Order within a pass.* Swaps first; collapses only when a full swap pass accepts nothing. Swaps move no nodes and therefore compose trivially with smoothing; collapses remove resolution and are the fallback. Splits participate where a bisection lowers the cavity energy (Remark 2); with a prescribed target, edges longer than $\sqrt{2}$ in the metric are the split candidates and edges shorter than $1/\sqrt{2}$ the collapse candidates, in addition to the edges of the high-energy cavities.
4. *Outer termination.* The coupled loop ends when the smoothing phase reports stagnation or convergence and the subsequent topology phase accepts zero operations, or when smoothing converges with $\max_e \hat{w}_e \leq \hat{w}_{\text{des}}$. Termination is progress-based, as in Omega_h; a large safety cap on outer iterations is retained for defense but is not the semantic terminator. When the goal is unreachable, the loop reports it and returns the best achieved mesh.

7 Question 2: energy monotonicity

The reading of the Omega_h source localizes its monotonicity exactly: strict improvement is guaranteed only where the strict filter is applied—swaps and sliver collapses—while refinement and size-driven collapse may degrade quality to the floor. Adapting the criterion to the EMS framework therefore means applying a strict-decrease filter, stated in energy, to *every* operation of the quality phase, and pairing it with a monotone smoothing phase. Both are stated against the intrinsic functional (2).

7.1 The acceptance gate

A topological operation replaces the elements C_{old} of a cavity by new elements C_{new} at fixed nodal positions $\mathbf{x}$ (for a split, the new node is placed and locally relaxed first; see Remark 3). The

operation is accepted if and only if

$$\sum_{e \in C_{\text{new}}} W_e(\mathbf{x}) \leq \sum_{e \in C_{\text{old}}} W_e(\mathbf{x}) - \delta, \quad \text{and} \quad \max_{e \in C_{\text{new}}} \hat{w}_e \leq \hat{w}_{\text{all}}, \quad (4)$$

with $\delta > 0$ a small relative margin (for example 10^{-8} times the old cavity energy) that precludes cycles of operations accepted at zero gain. The cavity *sum* replaces the cavity minimum of `Omega.h` deliberately: the sum is the restriction of the global functional to the cavity, so local acceptance implies global descent, which is the property the loop needs. The ceiling $\hat{w}_{\text{all}}$ plays the role of the allowed-quality floor; element inversion is excluded automatically because the Seth–Hill barrier sends $\hat{w}_e$ to infinity there.

Energy and geometric quality. The energy is the right quantity to accept an operation on, for three reasons. It is one functional for both phases, so the alternation cannot cycle and terminates (Proposition 1); a geometric criterion for the topology and an energy for the smoothing are two objectives, each phase can undo the other’s progress, and termination then rests on iteration caps, which is what `emend` does with its `maxOuterIterations`. It measures size, shape, and orientation at once: the scaled Jacobian and the mean ratio measure shape only, need a separate length band for size, and are blind to the orientation of an element relative to an anisotropic target unless the element is first mapped into the metric space. And it carries the inversion barrier. The geometric measures keep two roles. The first is the hard floor: the gate compares cavity sums, and a proposal could lower the sum while creating one element worse than the analysis code tolerates, so the second condition of (4) may be stated, in place of or in addition to $\hat{w}_{\text{all}}$, as the minimum scaled Jacobian or mean ratio the solver requires; both are cheap, and the geometric one is what the analysts already ask for, so reporting it keeps the results comparable with those of `improve_mesh`. The second is candidate selection through the length band, as stated above. One property of the energy to keep in view: it is not scale-free across elements of different target sizes, because the element energy is the density times the ideal volume. Thresholds and the floor are therefore on the density $\hat{w}_e$, while the cavity comparison is on the sums, which weight elements by their target volume as a global objective should.

7.2 The smoothing phase

The L-BFGS path of the smoothing solver already performs an Armijo backtracking line search on the energy, so each smoothing phase is monotone in its own frozen-target objective; the steepest-descent path currently backtracks on the residual norm instead and should use the energy merit for this loop. With re-baselining at phase start (Section 5), the phase objective agrees with $D(\mathcal{T}, \cdot)$ at the initial iterate. Because the phase objective holds its targets fixed while D is intrinsic, the phase is completed by a safeguard: evaluate D once at phase end, and roll the phase back if D increased. The safeguard costs one evaluation and is expected to be inert in practice; its purpose is to turn practical descent into provable descent.

Proposition 1 (Global descent and finite termination of the topological work). *Let the coupled loop consist of smoothing phases with the end-of-phase safeguard and topological operations accepted by the gate (4). Then the sequence of values of D sampled at phase boundaries and after each accepted operation is nonincreasing and bounded below by zero, hence convergent; each accepted operation decreases D by at least δ times the corresponding cavity energy at acceptance, so if δ is enforced against a fixed positive reference energy the number of accepted topological operations is finite.*

Proof. Only cavity elements change in a topological operation, so (4) implies D decreases by at least δ across it. The safeguard makes each smoothing phase nonincreasing in D by construction. A nonincreasing sequence bounded below converges, and a uniform positive decrement per accepted operation bounds their number. $\square$

The guarantee lives in exactly one place: a single acceptance routine implementing (4), called by every operator. No operator carries its own criterion.

Remark 2 (Splits participate honestly). *Because the equal-volume energy is scale-invariant, bisecting a badly stretched element into two better-shaped children genuinely lowers the cavity energy, and refinement needs no exemption from the gate (4). With a prescribed target the same holds for size-driven refinement and coarsening, since the metric-conditioned energy (1) penalizes the wrong size directly.*

Remark 3 (Local relaxation before the test). *For a split, place the new node at the edge midpoint—projected to the surface by `return_to_surface!` when the edge lies on the boundary—and run a few iterations of cavity-local smoothing with the cavity boundary fixed before evaluating (4). The cost is a handful of element evaluations; the benefit is that each operation is judged at its locally relaxed optimum rather than at an arbitrary initial placement, which raises the acceptance rate substantially. This is the natural advantage of an energy framework over the position-free acceptance tests of `Omega_h`, and it is the precise mechanism by which the alternation outperforms either method alone: each accepted operation unlocks relaxation that the previous topology forbade.*

8 Question 3: stagnation detection

The smoothing solver currently stops on a gradient-norm test with absolute and relative tolerances—a *convergence* criterion. Stagnation, the absence of significant further decrease, requires a criterion that is scale-free, tolerant of plateaus, and inexpensive. The smoothing phase should end at the first of the following three events, and report which one occurred.

1. *Windowed relative decrease.* Let W_n denote the phase objective at iteration n and let m be a window length (default $m = 10$). Stop when

$$W_{n-m} - W_n \leq \tau_{\text{stag}} (W_0 - W_n), \quad \tau_{\text{stag}} = 10^{-3} \text{ by default.} \quad (5)$$

In words: the decrease achieved over the last m iterations is a negligible fraction of the decrease achieved since the phase began. The test is invariant under affine rescaling of the energy and under problem size, and the window absorbs the short plateaus that quasi-Newton iterations produce near barrier-dominated configurations. Since Armijo backtracking makes $\{W_n\}$ monotone, the left side is nonnegative and the test is well posed.

2. *Convergence proper.* The existing projected-gradient test passes: the phase ended because the mesh is smooth, not because it is obstructed.
3. *Topological lock.* The line search backtracks to failure, or the non-positive-Jacobian guard trips repeatedly, while the projected gradient remains large. This is not convergence: it is the inversion barrier certifying that no continuous nodal motion can improve the mesh from the current configuration. It is the strongest possible trigger for the topology phase and should be detected and reported explicitly.

The soundness of (5) rests on the standard descent inequality: for a continuously differentiable objective with Armijo backtracking, the per-iteration decrease is bounded below by a multiple of the squared projected-gradient norm times the accepted step [5]. Sustained failure to decrease therefore certifies either a vanishing projected gradient (event 2 in disguise) or a collapsing step length against the barrier (event 3); the windowed test detects both with a single properly scaled quantity and no problem-dependent tuning.

Distinguishing the three events gives the outer loop its terminator at no additional cost: event 2 with $\max_e \hat{w}_e \leq \hat{w}_{\text{des}}$ means the loop is finished; events 1 and 3 call for a topology phase; a topology phase that accepts nothing after event 1 or 3 means the loop has reached the best this machinery can achieve, which is reported rather than concealed—the exact analogue of the explicit failure report of `Omega_h` when its quality goal is unreachable.

9 Architectural plan

The implementation follows the repository convention of paired files: `src/adapt.types.jl` for the option and cache types, `src/adapt.jl` for the methods. The steps are ordered so that each is independently testable, with the first coupled loop appearing at Step 4.

1. *Energy kernel on subsets.* Factor the per-element energy evaluation out of the assembly loop into a kernel callable on an arbitrary subset of elements at given nodal positions, with the metric-sampled ideal element of Section 5.2 for any element and the equal-volume re-baselined variant as the fallback, and register the density $\hat{w}_e$ as an Exodus element variable. Testable immediately against `model.stored.energy` on the existing `examples/ems` cases.
2. *Stagnation criterion.* Add the windowed test (5) and the lock detection to the solver stopping logic, returning a phase outcome (converged, stagnated, locked) rather than a Boolean, and switch the steepest-descent merit to the energy. Independently testable: the strongly distorted examples should report the lock outcome where they currently exhaust iterations.
3. *In-memory topology and adjacency.* A topology cache built once per topology phase from the Exodus-read connectivity: coordinates, per-block connectivity, and the three adjacencies the code base lacks—element-to-element across faces (by sorted-face matching), node-to-elements in compressed sparse row form, and an edge table mapping each edge to its incident elements. Boundary faces are identified as unpaired faces and cross-checked against the side sets; node-set and side-set membership is stored as per-entity flags so operations can update it. Four-node tetrahedra only, matching the existing restriction of `create_smooth_reference`.
4. *Edge swap and the acceptance gate; first coupled loop.* Implement the three-dimensional edge swap: extract the loop of elements around an edge (at most seven, following `Omega_h`), enumerate the tabulated triangulations of the surrounding polygon, and select the best; implement the single acceptance routine (4); wire the outer driver (smoothing phase, candidate collection, swap passes, repeat). Swaps move no nodes and need no state transfer, so this milestone alone should reproduce most of the observed benefit of the alternation.
5. *Edge collapse.* Sliver removal under the gate (4); collapses on the boundary are restricted to boundary edges with the surviving node kept on the surface through the `Surface` level-set machinery; nodes belonging to node sets are never removed; an orientation guard rejects collapses that would invert any element (already implied by the ceiling, but checked cheaply before the energy evaluation).

6. *Edge split and the size target.* Midpoint placement with surface projection for boundary edges, cavity-local relaxation per Remark 3, then the gate. With a prescribed target, edges longer than $\sqrt{2}$ or shorter than $1/\sqrt{2}$ in the metric join the candidate set, the target is used **unrestricted**, and a new element takes the metric at its centroid; these operations pass through the same gate as the others. The acceptance case is the tube of the anisotropic note with a radial size of a quarter of the wall, which smoothing alone cannot reach.
7. *Orchestration and output.* After each topology phase, rebuild the model, integrator, and solver through the existing mesh-replacement pattern of `src/swap.jl`, which avoids the pointer-aliasing hazard by construction. Write each adapted mesh as a fresh Exodus file in a numbered sequence: the Exodus format cannot change element counts within a file, so per-phase files are the practical route, and the replacement machinery already validates it. Input schema: an **adaptivity** block carrying $\hat{w}_{\text{des}}$, $\hat{w}_{\text{all}}$, the stagnation window and tolerance, the number of adjacency layers, and per-operator switches, mirroring **AdaptOpts**.
8. *Validation against the hand-off.* For the cube, the tube, and the strongly distorted examples, run the same input mesh and target through **improve_mesh** and through the native loop and compare the energy and scaled-Jacobian histograms of the two outputs; run **Omega_h** on the same Exodus files where a metric is prescribed and compare its length and quality histograms. The first milestone at which this comparison is meaningful is Step 4, since swaps alone, alternated with the existing smoother, are expected to show most of the benefit.
9. *Out of scope.* In-simulation adaptivity, which would invoke the driver from the evolution loop and transfer the solution state through `field_transfer.jl`, and quadratic elements, which are obtained by conversion after the loop. Both are noted here so that the data structures of Section 10 do not preclude them.

10 Data structures and design patterns

1. *Rebuild per phase, not incremental maintenance.* **Omega_h** rebuilds its topology arrays wholesale after each operator pass rather than patching them. The same pattern is right here: topology phases are rare relative to smoothing iterations, wholesale rebuild is simple and cache-friendly, and it eliminates the entire class of stale-adjacency defects. Within a pass, operations are applied through tombstones—dead elements and nodes are marked, new ones appended—and the arrays are compacted once at the end of the pass.
2. *One cavity abstraction, one gate.* Every operator produces a cavity proposal: the identifiers of the old elements, the proposed new connectivity, and optionally a new node. A single routine evaluates (4) and applies or discards the proposal. The monotonicity guarantee then has exactly one owner in the code.
3. *Sequential priority order now, independent sets later.* The Luby-style independent set of **Omega_h** exists for distributed parallelism. In serial, sorting the candidates by cavity energy in decreasing order and applying them sequentially is simpler and equally effective; a cavity that touches an element already modified in the current pass is deferred to the next pass, exactly as unselected **Omega_h** candidates wait for theirs. If the topology phase is ever threaded, adopt the independent set with the same comparator: the better cavity wins, with a deterministic identifier tiebreak.
4. *Exodus out of the hot path.* Connectivity is at present re-read from the database handle

inside the assembly loop. The topology cache of Step 3 should become the in-memory source of truth during adaptation, with Exodus demoted to input and output at phase boundaries. This change benefits plain EMS performance even before any topological operation exists.

5. *Outcome types over Booleans.* The solver reports converged, stagnated, or locked; each operator pass reports its number of accepted operations; the driver’s control flow is written entirely against these outcomes. This makes the progress-based termination of Section 6 auditable in the logs, in the same way that each Omega.h operator returns false exactly when its filter chain leaves no candidate.

11 Risks and open questions

Ranked by importance.

1. *The gate and the kernel.* The single functional of Section 5 is the load-bearing design decision. It should be validated by a small standalone prototype—a single cavity with hand-verified energies before and after a swap, a collapse, and a split, with the metric-sampled ideal and with the equal-volume fallback—before Step 4 is undertaken.
2. *Boundary collapses on curved surfaces.* The level-set projection and the energy test can conflict: a collapse that lowers the cavity energy may move the surviving node to a surface position that raises it again after projection. The gate must therefore be evaluated *after* projection, and boundary collapses should be expected to have a lower acceptance rate.
3. *Four-node tetrahedra only.* This is a decision, not a restriction to be lifted: the smoothing and the operators are written for four-node tetrahedra, and higher-order meshes are produced by conversion after the loop. The input documentation should say so.
4. *Energy thresholds across target sizes.* The element energy is the density times the ideal volume, so thresholds and the floor must be on the density and the cavity comparison on the sums (Section 7); a threshold on the element energy would favor large elements.
5. *Geometric floor versus energy gate.* An operation may lower the cavity energy while creating an element below the geometric floor the analysis requires; the floor is part of the gate for that reason, and its value should be taken from the analysis code, not from the smoother.

12 Summary

The coupled loop rests on one design decision and three algorithms. The decision is to use a single distortion functional for both phases: the metric-conditioned smoothing energy (1) when a size or metric field is prescribed, which is the common case for a preprocessing step and makes size-driven operations decreases of the same objective, and the intrinsic equal-volume functional (2) otherwise, so that in either case one quantity is meaningful across every smoothing step and every topological operation. The loop is a preprocessing step on four-node tetrahedra; it transfers no solution state, and higher-order meshes are obtained by conversion afterwards. Of the alternatives, wrapping Omega.h would reproduce the present staggered workflow with a new dependency, and the Sierra hand-off cannot be part of a public workflow; both remain useful as references for validation. The algorithms are: the transplanted Omega.h trigger logic—two-tier thresholds on the energy density, candidate dilation by adjacency layers, swaps before collapses, progress-based termination—; the acceptance gate (4), which strengthens the strict-improvement filter of Omega.h

from the cavity minimum to the cavity sum and thereby makes local acceptance imply global descent (Proposition 1); and the windowed stagnation test (5) with explicit detection of the topological lock, which supplies both the trigger for the topology phase and the terminator of the outer loop. Norma already provides the per-element energy field, the metric-space ideal element, the analytic surface constraints, and the mesh-replacement orchestration; the genuinely new construction is the in-memory topology cache with adjacency, the three cavity operators, and the single gate through which all of them pass.

References

- [1] D. A. Ibanez et al. Omega.h: reliable mesh adaptation. Sandia National Laboratories, https://github.com/sandialabs/omega_h.
- [2] D. A. Ibanez. *Conformal Mesh Adaptation on Heterogeneous Supercomputers*. PhD thesis, Rensselaer Polytechnic Institute, 2016.
- [3] L. A. Freitag, C. Ollivier-Gooch. Tetrahedral mesh improvement using swapping and smoothing. *International Journal for Numerical Methods in Engineering* 40 (1997) 3979–4002.
- [4] B. M. Klingner, J. R. Shewchuk. Aggressive tetrahedral mesh improvement. In: *Proceedings of the 16th International Meshing Roundtable*, Springer, 2007, 3–23.
- [5] J. Nocedal, S. J. Wright. *Numerical Optimization*, 2nd edition. Springer, 2006.
- [6] A. Mota, W. Sun, J. T. Ostien, J. W. Foulk III, K. N. Long. Lie-group interpolation and variational recovery for internal variables. *Computational Mechanics* 52 (2013) 1281–1299.